

Java Server Pages (JSP)

Java Server Pages (JSP) is a server-side programming technology that enables the creation of dynamic, platform-independent method for building Web-based applications. JSP have access to the entire family of Java APIs, including the JDBC API to access enterprise databases.

Java Server Pages (JSP) is a technology for developing web pages that support dynamic content which helps developers insert java code in HTML pages by making use of special JSP tags, most of which start with **<% and end with %>**

A Java Server Pages component is a type of Java Servlet that is designed to fulfill the role of a user interface for a Java web application. Web developers write JSPs as text files that combine HTML or XHTML code, XML elements, and embedded JSP actions and commands.

Using JSP, you can collect input from users through web page forms, present records from a database or another source, and create web pages dynamically. JSP tags can be used for a variety of purposes, such as retrieving information from a database or registering user preferences, accessing JavaBeans components, passing control between pages and sharing information between requests, pages etc.

Advantages of JSP

Java Server Pages often serve the same purpose as programs implemented using the Common Gateway Interface (CGI). But JSP offer several advantages in comparison with the CGI.

- **Performance is significantly better** because JSP allows embedding Dynamic Elements in HTML Pages itself instead of having a separate CGI files.
- **JSP are always compiled before it's processed by the server** unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.
- **Java Server Pages are built on top of the Java Servlets API**, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP etc.
- **JSP pages can be used in combination with Servlets that handle the business logic**, the model supported by Java Servlet template engines.

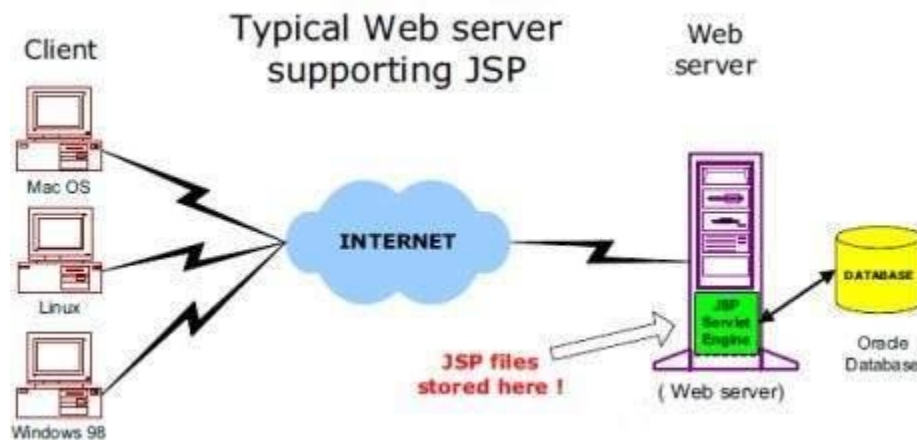
Finally, **JSP is an integral part of Java EE**, a complete platform for enterprise class applications. This means that JSP can play a part in the simplest applications to the most complex and demanding.

JSP - Architecture

The web server needs a JSP engine ie. Container to process JSP pages. The JSP container is responsible for intercepting requests for JSP pages. We use Apache Server which has built-in JSP container to support JSP pages development.

A JSP container works with the Web server to provide the runtime environment and other services a JSP needs. It knows how to understand the special elements that are part of JSPs.

Following diagram shows the position of JSP container and JSP files in a Web Application.



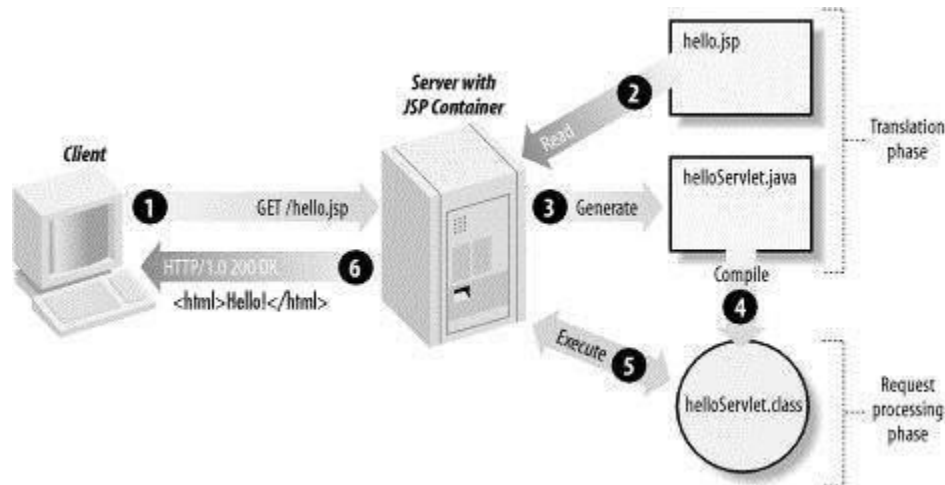
JSP Processing

The following steps explain how the web server creates the web page using JSP:

- As with a normal page, your browser sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by using the URL or JSP page which ends with **.jsp** instead of **.html**
- The JSP engine loads the JSP page from disk and converts it into Servlet content. This conversion is very simple in which all template text is converted to `println( )` statements and all JSP elements are converted to Java code that implements the corresponding dynamic behavior of the page.
- The JSP engine compiles the Servlet into an executable class and forwards the original request to a Servlet engine.

- A part of the web server called the Servlet engine loads the Servlet class and executes it. During execution, the Servlet produces an output in HTML format, which the Servlet engine passes to the web server inside an HTTP response.
- The web server forwards the HTTP response to your browser in terms of static HTML content.
- Finally web browser handles the dynamically generated HTML page inside the HTTP response exactly as if it were a static page.

All the above mentioned steps can be shown below in the following diagram:



Typically, the JSP engine checks to see whether a Servlet for a JSP file already exists and whether the modification date on the JSP is older than the Servlet. If the JSP is older than its generated Servlet, the JSP container assumes that the JSP hasn't changed and that the generated Servlet still matches the JSP's contents. This makes the process more efficient than with other scripting languages (such as PHP) and therefore faster.

So in a way, a JSP page is really just another way to write a Servlet without having to be a Java programming. Except for the translation phase, a JSP page is handled exactly like a regular Servlet

Configuring Apache Tomcat [Steps to create simple Servlet /JSP Web application using Java]

Step 1: Download and install latest version of JDK and Apache Tomcat from the Official site.

[\[http://www.oracle.com/technetwork/java/javase/downloads/index.html\]](http://www.oracle.com/technetwork/java/javase/downloads/index.html)

and

<https://tomcat.apache.org/>

Step 2: Set-up the CATALINA_HOME and CLASSPATH Environment Variable as:

JAVA_HOME = C:\Program Files\Java\jdk1.8.0_65;

PATH= C:\Program Files\Java\jdk1.8.0_65\bin;

CATALINA_HOME = C:\Program Files\Apache Software Foundation\Tomcat 9.0;

CLASSPATH= C:\Program Files\Apache Software Foundation\Tomcat 9.0\lib\servlet-api.jar;

Step 3: Create a Tomcat Director Structure as Stated below

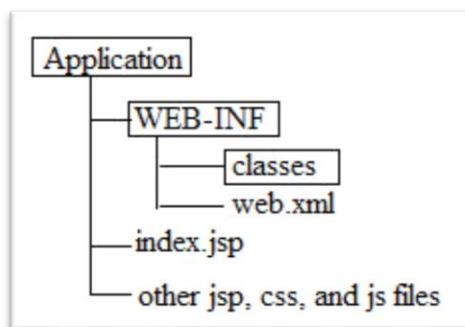
Step 4: Write the JSP and Servlet code to Compile and Place its Respective Folder as below.

[Note all the class files must be placed inside **classes** Directory under WEB-INF file]

Step 5: Write down the Mapping code for All the Servlet class under **web.xml**.

Step 6: Now write down the URL as http://localhost:8080/YOUT_ROOTFOLDERNAME/ in your favorite web browser. If you see output Congratulation you succeed otherwise make sure that all the mapping correctly

Folder structure if JSP and Servlet Server [tomcat]



Basic Hello World Program to print: **Greeting Message!!!**

```

<!DOCTYPE html>
<html>
<head>
<title>Hello World JSP Program!</title>
</head>
<body>
  <h1>
    <%
  
```

```

        out.println("Hello IT Professional, Welcome to Java
Server Page (JSP)");
    %>
</h1>
</body>
</html>

```

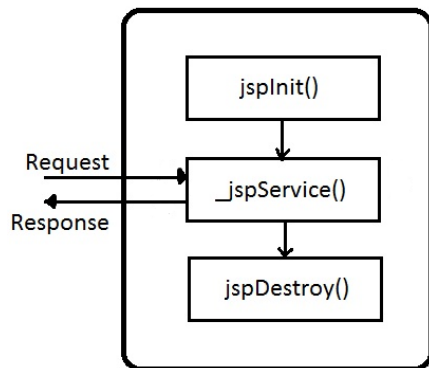
JSP Life-Cycle

JSP Life Cycle is defined as translation of JSP Page into servlet as a JSP Page needs to be converted into servlet first in order to process the service requests. The Life Cycle starts with the creation of JSP and ends with the disintegration of that.

Let's learn different phases of JSP Life Cycle

When the browser asks for a JSP, JSP engine first checks whether it needs to compile the page. If the JSP is last compiled or the recent modification is done in JSP, then the JSP engine compiles the page.

JSP Lifecycle is depicted in the below diagram.



Compilation process of JSP

JSP has the following three steps

- Parsing of JSP
- Turning JSP into servlet
- Compiling the servlet

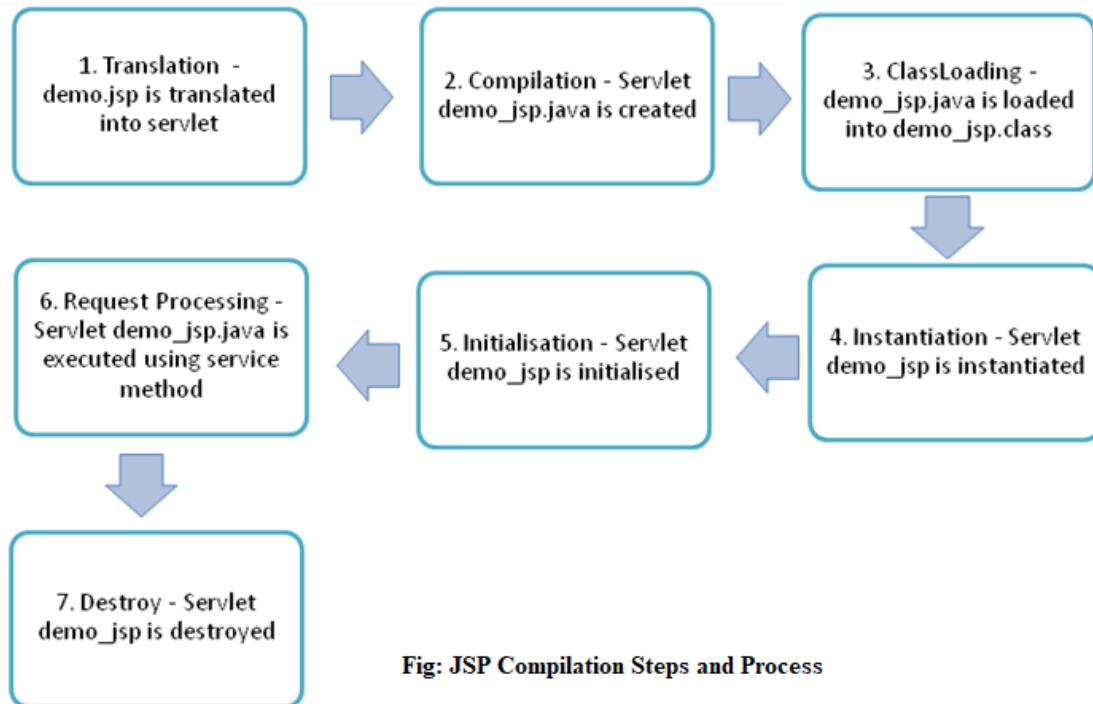


Fig: JSP Compilation Steps and Process

Following steps explain the JSP Compilation process

1. Translation of JSP page
2. Compilation of JSP page(Compilation of JSP page into _jsp.class)
3. Class loading (_jsp.java is converted to class file _jsp.class)
4. Instantiation(Object of generated servlet is created)
5. Initialization (_jspinit() method is invoked by container)
6. Request Processing(_jspservice() method is invoked by the container)
7. Destroy (_jspDestroy() method invoked by the container)

Let us have more detailed summary on the above points:

1. Translation of the JSP Page:

A `_Java_servlet` file is generated from a JSP source file. This is the first step of JSP life cycle. In translation phase, container validates the syntactic correctness of JSP page and tag files.

- The JSP container interprets the standard directives and actions, and the custom actions referencing tag libraries (they are all part of JSP page and will be discussed in the later section) used in this JSP page.
- In the above pictorial description, demo.jsp is translated to demo_jsp.java in the first step

- Let's take an example of "demo.jsp" as shown below:

Demo.jsp

```

1.  <html>
2.  <head>
3.  <title>Demo JSP</title>
4.  </head>
5.  <% int demvar=0;%>
6.  <body>
7.  Count is:
8.  <% out.println(demovar++); %>
9.  <body>
10. </html>

```

Code Explanation for Demo.jsp

Code Line 1: html start tag

Code Line 2: Head tag

Code Line 3 - 4: Title Tag i.e. Demo JSP and closing head tag

Code Line 5,6: Scriptlet tag wherein initializing the variable demo

Code Line 7 - 8: In body tag, a text to be printed in the output (Count is :)

Code Line 9: Scriptlet tag where trying to print the variable demovar with incremented value

Code Line 10-11: Body and HTML tags closed

Demo JSP Page is converted into demo_jsp servlet in the below code.

```

1  Public class demp_jsp extends HttpServlet{
2      Public void _jspervice(HttpServletRequest request, HttpServletResponse response)
3          Throws IOException, ServletException
4      {
5  PrintWriter out = response.getWriter();
6  response.setContentType("text/html");
7  out.write("<html><body>");
8  int demovar=0;
9  out.write("Count is:");
10 out.print(demovar++);
11 out.write("</body></html>");
12 }
13 }
14

```

Code explanation for Demo_jsp.java

Code Line 1: Servlet class demo_jsp is extending parent class HttpServlet

Code Line 2,3: Overriding the service method of jsp i.e. _jspervice which has HttpServletRequest and HttpServletResponse objects as its parameters

Code Line 4: Opening method

Code Line 5: Calling the method `getWriter()` of response object to get `PrintWriter` object (prints formatted representation of objects to text output stream)

Code Line 6: Calling `setContentType` method of response object to set the content type

Code Line 7: Using `write ()` method of `PrintWriter` object trying to parse html

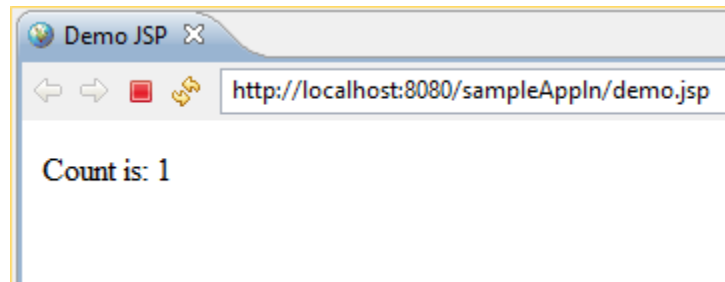
Code Line 8: Initializing `demovar` variable to 0

Code Line 9: Calling `write()` method of `PrintWriter` object to parse the text

Code Line 10: Calling `print()` method of `PrintWriter` object to increment the variable `demovar` from $0+1=1$. Hence, the output will be 1

Code Line 11: Using `write()` method of `PrintWriter` object trying to parse html

Output:



Here you can see that in the screenshot the Output is 1 because `demovar` is initialized to 0 and then incremented to $0+1=1$

In the above example,

`demo.jsp`, is a JSP where one variable is initialized and incremented. This JSP is converted to the servlet (`demo_jsp.class`) wherein the JSP engine loads the JSP Page and converts to servlet content.

When the conversion happens all template text is converted to `println()` statements and all JSP elements are converted to Java code.

This is how a simple JSP page is translated into a servlet class.

2. Compilation of the JSP Page

- The generated java servlet file is compiled into java servlet class
- The translation of java source page to its implementation class can happen at any time between the deployment of JSP page into the container and processing of the JSP page.
- In the above pictorial description `demo_jsp.java` is compiled to a class file `demo_jsp.class`

3. Classloading

- Servlet class that has been loaded from JSP source is now loaded into the container

4. Instantiation

- In this step the object i.e. the instance of the class is generated.
- The container manages one or more instances of this class in the response to requests and other events. Typically, a JSP container is built using a servlet container. A JSP container is an extension of servlet container as both the container support JSP and servlet.
- A JSP Page interface which is provided by container provides init() and destroy () methods.
- There is an interface HttpJSPPage which serves HTTP requests, and it also contains the service method.

5. Initialization

```
public void jspInit() { //initializing the code }
```

- _jspinit() method will initiate the servlet instance which was generated from JSP and will be invoked by the container in this phase.
- Once the instance gets created, init method will be invoked immediately after that
- It is only called once during a JSP life cycle, the method for initialization is declared as shown above

6. Request processing

```
void _jspservice(HttpServletRequest request HttpServletResponse response)
{
    //handling all request and responses
}
```

- _jspservice() method is invoked by the container for all the requests raised by the JSP page during its life cycle
- For this phase, it has to go through all the above phases and then only service method can be invoked.
- It passes request and response objects
- This method cannot be overridden
- The method is shown above: It is responsible for generating of all HTTP methods i.e GET, POST, etc.

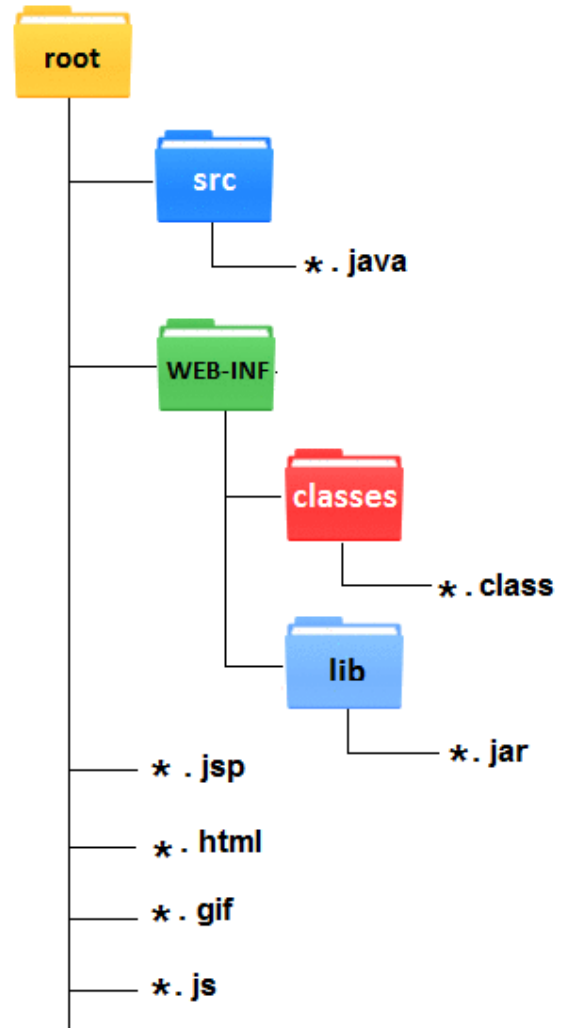
7. Destroy

```
public void _jspdestroy() { //all
clean up code }
```

- `_jspdestroy()` method is also invoked by the container
- This method is called when container decides it no longer needs the servlet instance to service requests.
- When the call to destroy method is made then, the servlet is ready for a garbage collection
- This is the end of the life cycle.
- We can override `jspdestroy()` method when we perform any cleanup such as releasing database connections or closing open files.

Directory structure of JSP

The directory structure of JSP page is same as Servlet. We contains the jsp page outside the WEB-INF folder or in any directory.



Because of jsp page contains outside of WEB-INF, JSP is a public file to the web application.

You can directly access these JSP pages from client system browser window.

JSP Syntax

A JSP Web page is created the same way you make an HTML file, except JSP code is added along with the HTML code. Additionally, instead of naming the file with an extension of **.HTM** or **.HTML**, the extension is **.JSP**

The JSP elements are **declarations**, **expressions**, **scriptlets**, and **directives**. Their own tags identify all of these elements.

- **JSP element declaration**

The declarations tag allows us to define **variables** and **methods**. The variables and methods are accessible from any **scriptlets** and **expressions** on the same page. Variable declarations are compiled as instance variables for a class that corresponds to the JSP page. Declarations are defined with the syntax:

```
<%!  
    Declarations  
%>
```

For example, the following defines an instance variable named **count** and a method named **incrementCount** () that increment the count variable:

```
<%!private int count = 0;  
  
    private void incrementCount() {  
        count++;  
    }%>
```

- **JSP Expression**

We can access variables defined in declarations with expression. The syntax to embed an expression is as follows:

```
<%=Expression%>
```

Expressions are embedded directly into the HTML. The Web browser will display the value of the expression in place of the tag. For example, we can output the value of the **count** variable in bold type with the following piece of HTML:

The value of count is: <%=count%>

- **JSP Scriptlet**

Blocks of Java code can be embedded in a **scriptlet**. The syntax for a **scriptlet** is as follows:

```
<%  
    Java code  
%>
```

If you wish to output HTML within a **scriptlet**, then this is done using **out.println()**, which is used in the same manner as **System.out.println()**. The variable **out** is already defined for us and is of type **javax.servlet.jsp.JspWriter**. Also note that **System.out.println()** will output to the console, which is useful for debugging purposes, while **out.println()** will output to the

browser. The following *scriptlet* invokes the *incrementCount()* method and then outputs the value in count :

```
<%
    out.println("The counter's value is " + count + "<br />");
    incrementCount();
%>
```

Following is a JSP page with a *declaration*, *expression*, and *scriptlet* that outputs text inside a header tag from levels 1 to 6. The identifier LASTLEVEL is declared as the last heading level that is to be displayed. LASTLEVEL is modified by static final, because it is intended as a constant. A loop inside the *scriptlet* outputs sample text for each level.

```
<html>
<title>Displaying Heading Tags with JSP</title>
<body>
    <%!private static final int LASTLEVEL = 6;%>
    <p>
        This page uses JSP to display Heading Tags from Level 1 to Level
        <%=LASTLEVEL%>
    </p>
    <%
        int i;
        for (i = 1; i <= LASTLEVEL; i++) {
            out.println("<H" + i + ">" + "This text is in Heading Level
" + i + "</H" + i + ">");
        }
    %>
</body>
</html>
```

- **JSP Directives**

Finally, let us introduce one more JSP tag, the *directive*. In general terms, directives instruct the compiler how to process a JSP program. Examples include the definition of our own tags, including the source code of other files, and importing packages. The syntax for directives is as follows:

```
<%@ page import="java.util.*,java.sql.*"%>
```

JSP Implicit Objects

JSP Implicit Objects are the Java objects that the JSP Container makes available to developers in each page and developer can call them directly without being explicitly declared. JSP Implicit Objects are also called pre-defined variables. JSP supports nine Implicit Objects which are listed below:

1. The **request object** is an instance of a ***javax.servlet.http.HttpServletRequest*** object. Each time a client requests a page the JSP engine creates a new object to represent that request.
2. The **response object** is an instance of a ***javax.servlet.http.HttpServletResponse*** object. Just as the server creates the request object, it also creates an object to represent the response to the client.
3. The **out** implicit object is an instance of a ***javax.servlet.jsp.JspWriter*** object and is used to send content in a response. The initial ***JspWriter*** object is instantiated differently depending on whether the page is buffered or not. Unlike the *PrintWriter* object, *JspWriter* throws *IOExceptions*.
4. The **session object** is an instance of ***javax.servlet.http.HttpSession*** and behaves exactly the same way that session objects behave under Java Servlets. The session object is used to track client session between client requests.
5. The **application object** is direct wrapper around the ***ServletContext*** object for the generated Servlet and in reality an instance of a ***javax.servlet.ServletContext*** object.
6. The **config object** is an instantiation of ***javax.servlet.ServletConfig*** and is a direct wrapper around the ***ServletConfig*** object for the generated servlet.
7. The **pageContext** object is an instance of a ***javax.servlet.jsp.PageContext*** object. The ***pageContext*** object is used to represent the entire JSP page. This object is intended as a means to access information about the page while avoiding most of the implementation details.
8. **Page** object is an actual reference to the instance of the page. It can be thought of as an object that represents the entire JSP page. The page object is really a direct synonym for the this object.
9. The **exception** object is a wrapper containing the exception thrown from the previous page. It is typically used to generate an appropriate response to the error condition.

Q. Write a simple JSP page to enter userName and display greeting message and add that username to message.

Soln:

user.jsp

```

<%
    if (request.getParameter("txtName") != null) {
        String name = request.getParameter("txtName");
        out.println("Welcome, " + name);
    }
%>
<!DOCTYPE html>
<html>
<head>
<title>JSP page</title>
</head>
<body>
    <form method="post" action="abc">
        Username <input type="text" name="txtName" /> <input
type="submit"
        value="GO" />
    </form>
</body>
</html>

```

Q. Create a login form in JSP file and when the user clicks on login button the username and password should be checked by another jsp and if username is “nast” and password is “bcA@7th15” display greeting message otherwise printed invalid details.

Soln:- *login.jsp [This is simple JSP UI page]*

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Login</title>
</head>
<body>
    <form method="POST" action="authentication.jsp">
        <table>
            <tr>
                <td>Username:</td>
                <td><input type="text" name="txtUserName"
required="required"
                placeholder="@Username" /></td>
            </tr>
            <tr>
                <td>Password:</td>
                <td><input type="password" name="txtPassword"

```

```

                                required="required" placeholder="@Password"
/></td>
                                </tr>
                                <tr>
                                <td><input type="reset" name="btnReset"
value="Reset" /></td>
                                <td><input type="submit" name="btnLogin"
value="Login" /></td>
                                </tr>
                                </table>
                                </form>
</body>
</html> Output:

```

The screenshot shows a web form with two input fields. The first field is labeled 'Username:' and contains the text 'admin'. The second field is labeled 'Password:' and contains four dots '....'. Below the password field are two buttons: 'Reset' and 'Login'.

authenticator.jsp

```

<%
    String name = request.getParameter("txtUserName");
    String password = request.getParameter("txtPassword");
    if (name.equals("nast") && password.equals("bcA@7th15")) {
        out.println("<h1>Welcome to JSP World!</h1>");
    } else {
        out.println("<h4>Sorry, Username and password doest not
match");
    }
%>

```

Q. Create a simple JSP form to take TWO integer number and when the user clicks on ADD button the sum of two numbers will be displayed in the other jsp Page.

Soln:-

addition.jsp

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Addition</title>

```

```

</head>
<body>
<h3>Addition!</h3>
<form method="POST" action="calc.jsp">
    <table>
        <tr>
            <td>First Number:</td>
            <td><input type="number" name="txtN1"
required="required"
placeholder="@First Number" /></td>
        </tr>
        <tr>
            <td>Second Number:</td>
            <td><input type="number" name="txtN2"
required="required" placeholder="@Second
Number" /></td>
        </tr>
        <tr>
            <td>&nbsp;   </td>
            <td><input type="submit" name="btnAdd"
value="ADD" /></td>
        </tr>
    </table>
</form>
</body>
</html>

```

calc.jsp

```

<%
    if (request.getParameter("btnAdd") != null) {
        Double n1 =
Double.parseDouble(request.getParameter("txtN1"));
        Double n2 =
Double.parseDouble(request.getParameter("txtN2"));
        out.print("<h4>The sum of " + n1 + " and " + n2 + " is:" +
(n1 + n2));
    }
%>

```


Q. Write a program to find profit or loss when user supply cost and sales price

Soln:-

profitloss.jsp

```
<!DOCTYPE html>
<html>
<head>
<title>Profit Loss Calculator!</title>
</head>
<body>
    <h3>Profit Loss Calculator!</h3>
    <form method="POST">
        <table>
            <tr>
                <td>Cost Price:</td>
                <td><input type="number" name="txtCp"
required="required"
placeholder="Cost Price" /></td>
            </tr>
            <tr>
                <td>Sales Price:</td>
                <td><input type="number" name="txtSp"
required="required"
placeholder="Sales Price" /></td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td><input type="submit" name="btnCalcualte"
value="CALCULATE" /></td>
            </tr>
        </table>
    </form>

    <%
        if (request.getParameter("btnCalcualte") != null) {
            Double cp = Double.parseDouble(request.getParameter("txtCp"));
            Double sp = Double.parseDouble(request.getParameter("txtSp"));
            Double result = sp - cp;
            Double percent = (result / cp) * 100;
            if (result > 0) {
                out.print("<h3>Profit Amount is : " + result + "</h3>");
                out.println("<h4>Profit: " + percent + "%</h4>");
            } else {
                out.print("<h3>Loss Amount is : Rs. " + Math.abs(result) +
"</h3>");
            }
        }
    %>
```

```

        out.println("<h4>Loss: " + Math.abs(percent) + "%</h4>");
    }
}
%>
</body>
</html>

```

Q. Write a program to authenticate an user from database

User.java

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class User {

    public static final String DRIVER = "com.mysql.jdbc.Driver";
    public static final String URL = "jdbc:mysql://192.168.10.20:3306/db";
    public static final String USER_NAME = "admin";
    public static final String PASSWORD = "pass";

    private static Connection getConnection() {
        Connection conn = null;
        try {
            Class.forName(DRIVER);
            System.out.println("Driver Loaded Successfully...");
            conn = DriverManager.getConnection(URL, USER_NAME,
PASSWORD);
            System.out.println("Database connected successfully....");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
        return conn;
    }

    public static Boolean isAuthenticated(String userName, String password)
{
    Boolean authenticated = false;
    ResultSet rs = null;
    String sql = "SELECT id from user_registration WHERE user_name='"
+ userName + "' AND password='" + password
        + "'";
    Connection conn = getConnection();
    try {
        Statement stmt = conn.createStatement();

```

```

        rs = stmt.executeQuery(sql);
        while (rs.next()) {
            if ((rs.getInt("id")) > 0) {
                authenticated = true;
            }
        }
    } catch (SQLException e) {
        System.out.println(e);
    }
    return authenticated;
}

public static void main(String[] args) {
    if(User.isAuthenticated("abc", "xyz")){
        System.out.println("User is Authenticated");
    }else{
        System.out.println("User is not Registered in database");
    }
}
}

```

Sending an Email Using JavaMail

Step1: Download and Include activation.jar and mail.jar file to Java Build Path

Step 2: Design a Sender class as:

```

import java.util.*;
import javax.mail.*;
import javax.mail.internet.*;

public class Sender {

    final static String EMAIL = "javatrainingnepal@gmail.com";
    final static String PASSWORD = "BE@2018_nast";
    private static int RESULT = 0;

    public static int send(String to, String subject, String body){
        try {
            Properties props = System.getProperties();
            props.setProperty("mail.transport.protocol", "smtp");
            props.setProperty("mail.host", "smtp.gmail.com");
            props.put("mail.smtp.auth", "true");
            props.put("mail.smtp.port", "25");
            props.put("mail.debug", "true");
            props.put("mail.smtp.socketFactory.port", "25");
            props.put("mail.smtp.socketFactory.class",
                "javax.net.ssl.SSLSocketFactory");
            props.put("mail.smtp.socketFactory.fallback", "false");

```

```

        Session emailSession = Session.getInstance(props,
            new javax.mail.Authenticator() {
                protected PasswordAuthentication
getPasswordAuthentication() {
                    return new
PasswordAuthentication(EMAIL,PASSWORD);
                }
            });
        emailSession.setDebug(true);
        Message message = new MimeMessage(emailSession);
        message.setFrom(new InternetAddress(EMAIL));

        message.setRecipients(Message.RecipientType.TO,
            InternetAddress.parse(to));
        message.setSubject(subject);
        message.setText(body);

        Transport transport = emailSession.getTransport("smtps");
        transport.connect("smtp.gmail.com", EMAIL, PASSWORD);
        transport.sendMessage(message, message.getAllRecipients());
        RESULT = 1;

    } catch (MessagingException e) {
        RESULT = 0;
    }
    return RESULT;
}
}

```

Step3: Design a client page

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-
1">
<title>Simple Email Client Application</title>
<link href="css/bootstrap.css" />
</head>
<body>
    <div>
        <h2>Email Client Application</h2>
        <h4>
            <%
                if (request.getParameter("status") != null) {
                    String status =
request.getParameter("status");

```

```

        if (status.equals(new String("0"))) {
            out.print("unable to send");
        } else {
            out.print("sent");
            out.print("<meta http-equiv='refresh'
content='2;url=send-email.jsp' />");
        }
    }
%>
</h4>
<form method="post" action="send.jsp">
    <table>
        <tr>
            <td>To</td>
            <td><input type="email" name="txtTo"
/></td>
        </tr>
        <tr>
            <td>Subject</td>
            <td><input type="text" name="txtSubject"
/></td>
        </tr>
        <tr>
            <td>Message</td>
            <td><textarea
name="txtMessage"></textarea></td>
        </tr>
        <tr>
            <td>&nbsp;</td>
            <td><input type="submit" name="btnSend"
value="SEND" /></td>
        </tr>
    </table>
</form>
</div>
</body>
</html>

```

Step4: Design and Implement Email Send Logic

```

<%@page import="email.Sender"%>
<%
    String to = request.getParameter("txtTo");
    String subject = request.getParameter("txtSubject");
    String body = request.getParameter("txtMessage");
    int status = Sender.send(to, subject, body);

```

```
%> response.sendRedirect("send-email.jsp?status=" + status);
```